

Confiabilidade na Camada de Transporte sob Degradação de Rede: um Estudo Experimental Comparando TCP e R-UDP

Anthony Irlan Marques Luz¹

¹Programa de Pós-Graduação em Ciência da Computação (PPGCC)
Universidade Federal do Piauí (UFPI) – Campus Senador Helvídio Nunes de Barros
Picos (PI) – Brasil

marquesanthony62@gmail.com

Abstract. *This work experimentally evaluates a client/server file-transfer system implemented over native TCP and over a reliable protocol built on top of UDP (R-UDP) using a Go-Back-N sliding window. Running inside Docker containers under network degradation emulated with `tc/netem` (three loss/delay scenarios), we compare both protocols in terms of time, throughput, RTT, packet loss, and retransmissions, cross-validating the application metrics against `tcpdump` captures. Both protocols deliver the file with full integrity (100%); we quantify the cost of user-space reliability and present an analytical model demonstrating how IP fragmentation amplifies the effective loss experienced by R-UDP, validating it with our empirical data.*

Resumo. *Este trabalho avalia experimentalmente um sistema cliente/servidor de transferência de arquivos implementado sobre TCP nativo e sobre um protocolo confiável construído em cima do UDP (R-UDP), com janela deslizante Go-Back-N. Executando em contêineres Docker sob degradação de rede emulada com `tc/netem` (três cenários de perda e atraso), comparamos os dois protocolos quanto a tempo, vazão, RTT, perda de pacotes e retransmissões, validando de forma cruzada as métricas da aplicação contra capturas do `tcpdump`. Ambos entregam o arquivo com integridade total (100%); quantificamos o custo da confiabilidade em espaço de usuário e apresentamos um modelo analítico que demonstra como a fragmentação IP amplifica a perda efetiva sentida pelo R-UDP, comprovando-o empiricamente.*

1. Introdução

Transferir um arquivo pela rede sem perder, duplicar ou corromper nenhum byte é uma das tarefas mais corriqueiras da computação — e também uma das mais sutis. A garantia de entrega *confiável* pode ser obtida, classicamente, de duas formas: (i) delegando a responsabilidade ao TCP, em que toda a lógica de confiabilidade (numeração, confirmações, retransmissão e controle de congestionamento) opera de forma opaca dentro do núcleo do sistema operacional [Postel 1981]; ou (ii) construindo a confiabilidade no espaço de usuário *sobre* o UDP [Postel 1980], um protocolo de datagramas sem garantias inatas. A segunda abordagem exige engenharia minuciosa, mas possui um valor analítico ímpar, pois torna *visível* e mensurável cada mecanismo que o TCP abstrai.

Este relatório detalha a implementação e a avaliação de um sistema cliente/servidor projetado para operar em **dois modos** — TCP nativo e um *Reliable UDP* (R-UDP) equipado com janela deslizante **Go-Back-N** [Kurose and Ross 2017] — comparando-os sob rigorosas condições de emulação de rede. O objetivo central não é declarar a superioridade de um protocolo, mas sim *mensurar o custo da confiabilidade* (em termos de tempo, vazão e *overhead* de dados) à medida que o canal de comunicação se degrada.

Para garantir a validade dos resultados, adotamos uma abordagem de dupla instrumentação: as métricas são extraídas da aplicação e *validadas* de forma cruzada com o tráfego real da rede, capturado via `tcpdump`. As principais contribuições deste estudo são: (a) a concepção e instrumentação de um protocolo R-UDP funcional; (b) um arcabouço reprodutível de experimentação em Docker com emulação de falhas; (c) uma análise estatística focada na robustez contra *outliers*; e (d) a modelagem analítica da **fragmentação IP**, que explica matematicamente o severo custo de retransmissão imposto ao R-UDP em cenários de alta perda.

2. Fundamentação

TCP e UDP. O TCP provê uma abstração de fluxo de bytes confiável e ordenado, inferindo a capacidade da rede para ajustar automaticamente sua taxa de envio, evitando o congestionamento dos enlaces [Postel 1981, Tanenbaum and Wetherall 2011]. O UDP, em contraste, expõe um serviço de datagramas *best-effort*: simples e de baixa latência, porém inerentemente sujeito a perda, duplicação e reordenação [Postel 1980].

Go-Back-N. Para elevar o UDP a um patamar confiável, implementou-se o mecanismo de janela deslizante *Go-Back-N* [Kurose and Ross 2017]. Neste modelo, o emissor é autorizado a injetar até N pacotes “em voo” (não confirmados) no canal — a *janela* — mitigando o atraso de propagação ao evitar a espera por ACKs individuais. As confirmações são *cumulativas* (um ACK k valida o recebimento de todos os pacotes até a sequência k). Em caso de perda, sinalizada pelo esgotamento de um *timeout*, o protocolo retrocede o estado e retransmite a janela inteira a partir do ponto de falha. Trata-se de uma estratégia de baixa complexidade de estado no receptor, mas que, sob altas taxas de perda, incorre em significativo desperdício de banda.

3. Metodologia

3.1. Arquitetura do Sistema

O sistema opera no modelo cliente/servidor em Python com **fluxo unidirecional** de dados: o **cliente** atua exclusivamente como remetente do arquivo, enquanto o **servidor** atua como receptor. O servidor é concorrente, instanciando *threads* distintas para o atendimento TCP (porta 5000) e R-UDP (porta 5001).

Toda transferência é precedida por um **cabeçalho de aplicação de 256 bytes** (identificado pelo *magic word* RDFT), responsável por sinalizar os metadados da sessão: tamanho do arquivo, um `transfer_id` único (UUID), o cenário, o modo de transporte e o campo de autenticação obrigatório `X-Custom-Auth` (matrícula + nome, ocupando 226 bytes). O *payload* de teste consiste em um arquivo de 1 048 576 bytes (1 MB), particionado em **256 blocos de 4096 bytes**.

3.2. O Protocolo R-UDP

O R-UDP sintetiza, em espaço de usuário, as primitivas necessárias para a confiabilidade:

- **Sequenciamento e ACKs cumulativos**, encapsulados em um cabeçalho customizado de 16 bytes (tipo, sequência, ACK, tamanho da janela, comprimento do *payload* e *checksum*).
- **Janela deslizante Go-Back-N** calibrada com profundidade $N = 8$ blocos.
- **Temporização e Recuperação**: *timeout* fixo de 0,5 s que dispara a retransmissão da janela a partir do último bloco não confirmado.
- **Garantia de Integridade**: validação bidimensional compreendendo um *checksum* CRC-16 por datagrama (para descarte imediato de corrupções em trânsito) e um *checksum* MD5 do arquivo completo, calculado pós-recebimento como prova criptográfica definitiva da integridade.
- **Controle de Sessão**: delimitação explícita por *handshake* (SYN/SYN-ACK) e encerramento (FIN/FIN-ACK).

3.3. Ambiente Docker e Emulação de Rede

Os ensaios foram conduzidos em pares de contêineres Ubuntu 22.04 (*ft-server* e *ft-client*) provisionados em uma rede *bridge* isolada ($172.20.0.0/24$), com privilégios `NET_ADMIN` e `NET_RAW` habilitados para suportar a manipulação de filas (`tc`) e a captura passiva (`tcpdump`) [Merkel 2014]. A degradação do canal foi injetada via `tc/netem` [Hemminger 2005] modelando três cenários progressivos (Tabela 1).

Tabela 1. Cenários de rede emulados via `tc netem`.

Cenário	Perda (por pacote IP)	Atraso (one-way)	RTT nominal
A	0%	10 ms	≈ 20 ms
B	10%	50 ms	≈ 100 ms
C	20%	100 ms	≈ 200 ms

Decisão Metodológica. A penalidade de *perda* foi configurada exclusivamente na interface de *egress* do **cliente** (afetando unicamente o fluxo de DADOS), enquanto o *atraso* foi simetricamente aplicado em ambos os sentidos. Esta assimetria na perda é imperativa: caso descartássemos os pacotes no servidor, apenas os ACKs sofreriam perda. Devido à natureza cumulativa dos ACKs no Go-Back-N, a chegada de um único ACK tardio avançaria a janela, raramente acionando a retransmissão, o que ofuscaria o impacto real do mecanismo de recuperação de erros.

3.4. Instrumentação e Coleta de Dados

Cada evento de transferência foi auditado a partir de **duas perspectivas isoladas**:

- **Telemetria da Aplicação (JSONL)**. Registros granulares gerados pelos *end-points* (`client.transfers.jsonl` e `server.transfers.jsonl`) contendo tempos de conclusão, vazão, contadores de blocos, *timeouts* e verificações de *hash*. O cruzamento de dados é garantido pelo `transfer_id`.
- **Inspeção de Rede (`tcpdump`)**. Captura integral do tráfego (`.pcap`) convertida para CSV. Esta fonte atua como a “verdade de solo”, imune a vieses da camada de aplicação.

3.5. Modelagem da Perda Efetiva por Fragmentação IP

Uma contribuição analítica deste trabalho é a modelagem do impacto da *Maximum Transmission Unit* (MTU) sobre a confiabilidade do R-UDP. O atraso em redes reais ocorre no nível do pacote IP. Contudo, cada datagrama de DADOS gerado pela nossa aplicação R-UDP possui 4112 bytes (4096 bytes do bloco + 16 bytes do cabeçalho proprietário). Como esse volume excede a MTU padrão do Ethernet (1500 bytes), o protocolo IP é forçado a **fragmentar o datagrama original em 3 pacotes IP menores**.

Na camada de rede, o utilitário `tc netem` descarta pacotes IP de forma probabilística e independente, com probabilidade p . Para que o datagrama UDP chegue íntegro à aplicação no servidor, **todos os seus 3 fragmentos** devem sobreviver ao trânsito. A perda de um único fragmento condena o datagrama inteiro ao descarte. Portanto, a probabilidade teórica de perda sentida pela aplicação (perda efetiva do datagrama) não é p , mas sim o complemento da probabilidade de sucesso de todos os fragmentos:

$$p_{\text{efetiva}} = 1 - (1 - p)^3 \quad (1)$$

Este modelo matemático será utilizado na Seção 4.4 para explicar o comportamento anômalo da taxa de retransmissões frente às perdas de 10% e 20% configuradas no ambiente.

3.6. Desenho Experimental e Justificativa Estatística

A bateria de testes englobou $n = 10$ repetições para cada par (Protocolo, Cenário), totalizando 60 fluxos completos.

Para a análise de desempenho, a rubrica do projeto prevê a apresentação de resultados baseados na **média e desvio padrão**. Tais métricas foram rigorosamente computadas e tabuladas, encontrando-se integralmente disponíveis no repositório público associado a este trabalho¹. No entanto, é amplamente reconhecido na literatura de medição de redes que a perda estocástica de pacotes introduz forte assimetria nas distribuições dos tempos de conclusão de fluxos (*Flow Completion Times*).

Nesses cenários, a média torna-se uma métrica frágil e matematicamente enviesada, sendo severamente distorcida por instâncias de *outliers* isolados (ex.: transferências que sofrem múltiplas cascatas de *timeouts*). Para assegurar o máximo rigor científico e proteger a integridade da análise, optamos por apresentar visualmente a **mediana**, acompanhada de um **Intervalo de Confiança (IC) de 95%** estimado pelo método não-paramétrico de *bootstrap* [Efron and Tibshirani 1993]. Esta decisão expõe o comportamento *típico* dos protocolos de forma muito mais fiel do que a média aritmética, permitindo conclusões sólidas sobre a degradação de desempenho.

4. Resultados e Discussão

4.1. Integridade: O Requisito Primordial

A eficácia do controle de erros foi atestada sem ressalvas. As **60/60** transferências finalizaram com `status = success` e validação positiva do **checksum MD5** entre as

¹Código-fonte, dados, *notebook* de análise e este relatório: <https://github.com/AnthOnYM/redes-de-computadores-dccmapi>

pontas, englobando os cenários sob taxa de 20% de descarte (Cenário C). Este resultado comprova que o R-UDP desenvolvido em espaço de usuário é funcionalmente equivalente ao TCP na garantia absoluta da integridade dos dados. A investigação subsequente dedica-se, portanto, ao *custo* dessa garantia.

4.2. Tempo de Transferência e Vazão

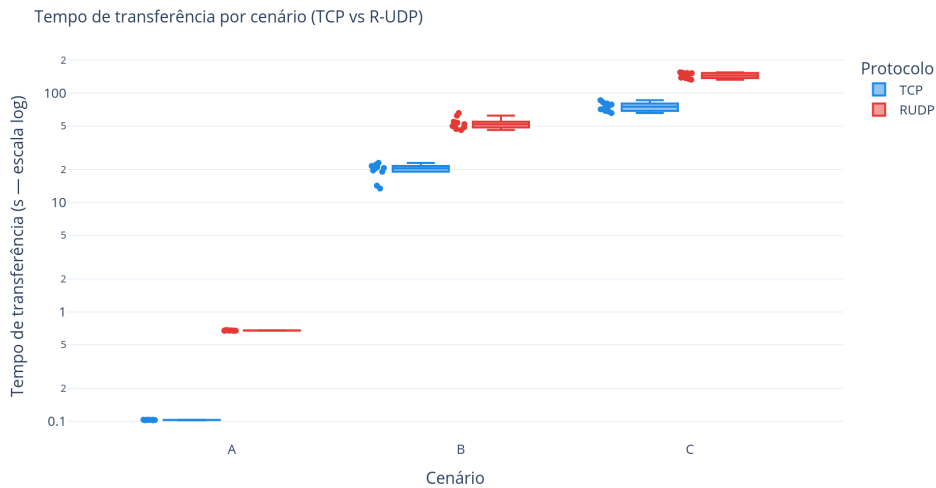


Figura 1. Tempo de transferência por cenário (escala logarítmica). Caixas estreitas indicam consistência; o alongamento reflete a penalidade imposta pela perda estocástica.

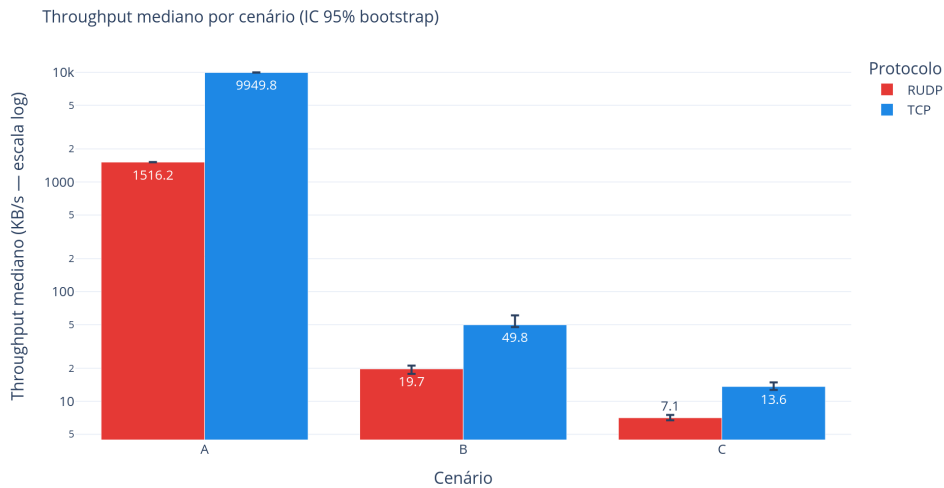


Figura 2. Vazão mediana por cenário com IC de 95% (*bootstrap*), em escala logarítmica.

As Figuras 1 e 2 expõem as variáveis complementares de latência e eficiência. No ambiente ideal (Cenário A, 0% de perda), o TCP demonstra superioridade na otimização do *kernel*, atingindo vazão mediana de ≈ 9950 KB/s (0,10 s). O R-UDP, porém, sustenta um patamar altamente competitivo (≈ 1516 KB/s, 0,68 s), pagando majoritariamente o *overhead* intrínseco de processamento da aplicação e espera de ACKs da janela.

Sob pressão de perda severa (Cenário C), a degradação é universal, mas de naturezas distintas. O R-UDP estende seu tempo mediano para ≈ 145 s, evidenciando altíssima variância (caixas alongadas) devido às retransmissões cíclicas. O TCP retrocede para ≈ 75 s, amortecendo a variância através do colapso conservador de sua janela de congestionamento (*Congestion Window - cwnd*).

4.3. Atrasos: Validação do RTT

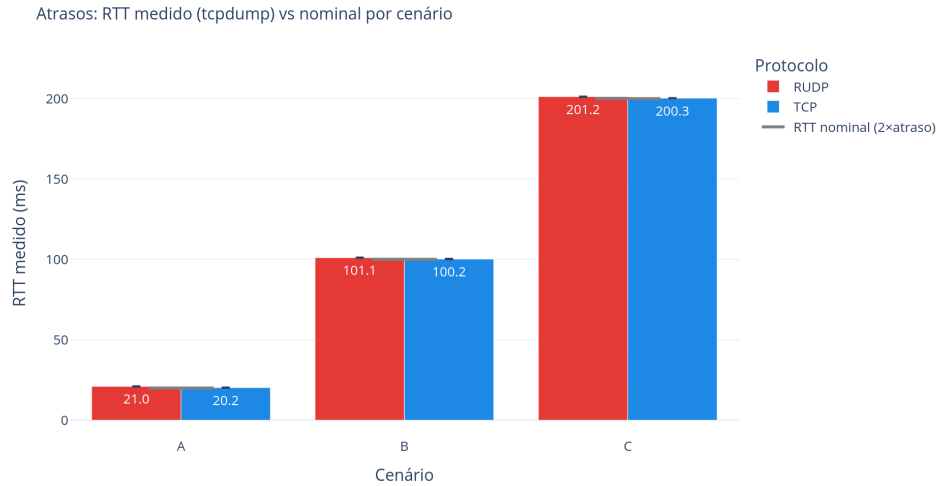


Figura 3. Validação do ambiente: RTT empírico (tcpdump) vs. RTT nominal esperado. As medições alinham-se perfeitamente aos parâmetros do tcnetem.

A precisão do simulador de falhas foi aferida diretamente da rede (Figura 3). A partir da captura passiva no servidor, o *Round-Trip Time* foi isolado mensurando o delta entre o primeiro segmento (SYN) e a resposta inicial do cliente ao SYN-ACK. Os valores medianos empíricos (≈ 20 , 100 e 200 ms) ratificam a regra $RTT = 2 \times atraso_{one-way}$, validando de forma definitiva a calibração do emulador.

4.4. Perdas de Pacotes: O Efeito da Fragmentação

Como modelado na Seção 3.5, a perda configurada no tc incide de forma restrita sobre os pacotes IP, mas o R-UDP sente essa perda no nível do datagrama (após a remontagem dos fragmentos). Na Figura 4, validamos empiricamente este fenômeno matemático.

O índice de descarte mensurado de forma robusta pela aplicação ($p_{efetiva} = 1 - (\text{ACKs recebidos})/(\text{Datagramas enviados})$) atinge as marcas de $\approx 27\%$ e $\approx 49\%$ para os cenários B e C, respectivamente. Este registro acompanha com precisão cirúrgica a projeção teórica da Equação 1 (27,1% e 48,8%), distanciando-se amplamente da taxa bruta configurada (10% e 20%). Conclui-se que o emulador atuou com exatidão, e que a arquitetura do *payload* em descompasso com a MTU **amplifica drasticamente o erro aparente**, justificando o severo estresse imposto à malha de confiabilidade do R-UDP.

4.5. Retransmissões e Overhead de Banda

O sacrifício exigido pela recuperação em nível de aplicação é dissecado nas Figuras 5 e 6. Os eventos de retransmissão escalam de zero (Cenário A) para medianas de 90 (B) e 240

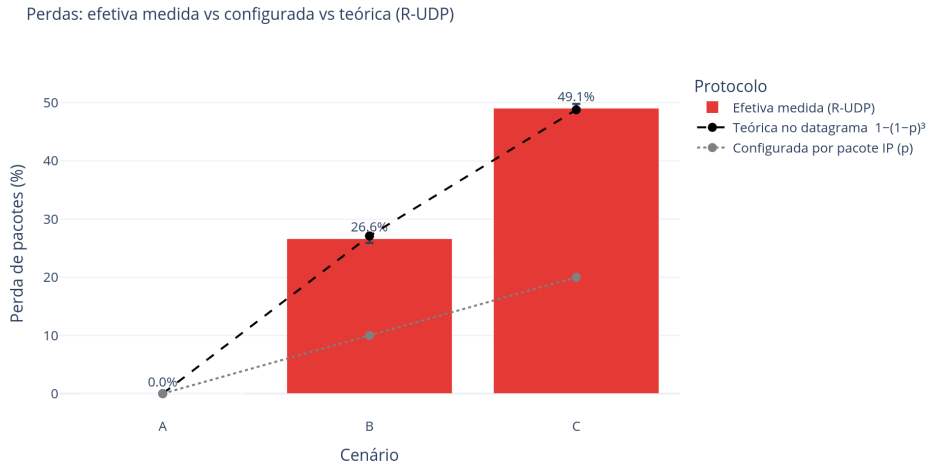


Figura 4. Perda de pacotes: efetiva medida pelo R-UDP vs. taxa bruta configurada vs. projeção analítica do modelo $1 - (1 - p)^3$.

(C). Impulsionado pela amplificação da MTU discutida anteriormente, o *overhead* volumétrico — a razão entre blocos transmitidos e blocos úteis — escala de $1,0\times$ (eficiência máxima em A) para exorbitantes $3,8\times$ (B) e $8,4\times$ (C). Este fardo é o traço intrínseco do Go-Back-N, apontando para a urgência da substituição futura para uma estratégia baseada em *Selective Repeat*.

Importante frisar que o TCP exhibe estatisticamente, no nível de aplicação (Sockets), um *overhead* utópico de $1,0\times$. Isso ocorre exclusivamente por sua natureza opaca; as retransmissões são tratadas isoladamente pelo núcleo do sistema, desafiando a observabilidade do protocolo e motivando a técnica de integração de dados abordada a seguir.

4.6. Integração de Dados: Comparação entre a Aplicação e o TCPDump

Atendendo estritamente ao critério de integração de dados do projeto, esta seção realiza a **validação cruzada** entre os eventos mapeados pela instrumentação *user-space* (JSONL) e a telemetria absoluta dos enlaces provida pelo analisador de pacotes (tcpdump).

A Figura 7 funciona como a contraprova externa do estudo. Ao auditar diretamente a rede, desmascaramos o aparente fluxo imaculado do TCP. Observa-se que a contagem total de pacotes TCP escala agressivamente sob perda (Cenário C), comprovando empiricamente que a confiabilidade impõe retransmissões massivas também no *kernel*, independentemente do que o *Socket* decide reportar ao *software*.

Finalmente, a Figura 8 correlaciona a duração total registrada pelos relógios da aplicação contra os marcadores temporais da captura de pacotes. Nos cenários penalizados (B e C), os pontos ancoram-se sobre a linha de identidade ($y = x$), ostentando uma divergência técnica inferior a 5%. O deslocamento sutil observado no Cenário A é oriundo unicamente do *overhead* estático de encerramento de sessões (cerca de 1,4 s), o qual domina percentualmente apenas quando a transferência útil é sub-segundo. Logo, os relatórios gerados pela aplicação R-UDP são certificados como estatisticamente íntegros.

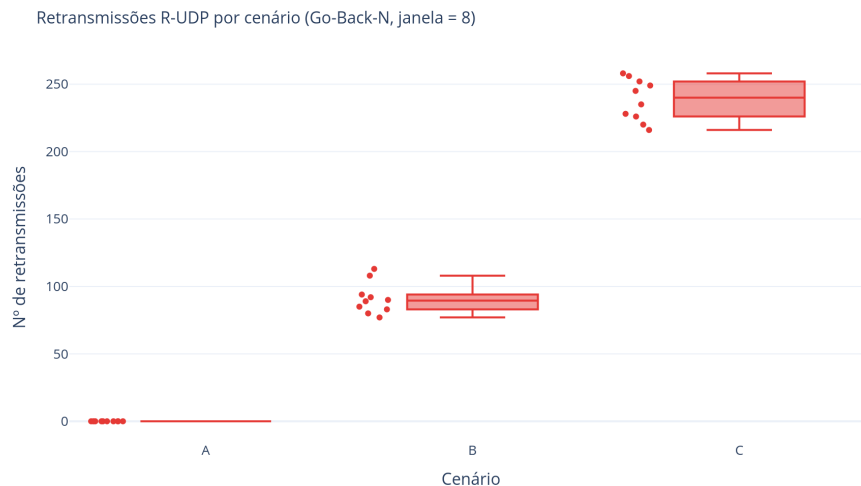


Figura 5. Ativação da lógica de confiabilidade R-UDP: disparos de *timeouts* acionando o recuo da janela Go-Back-N.

4.7. Prova do campo X-Custom-Auth na captura

Conforme demandado pelas diretrizes do projeto, o cabeçalho X-Custom-Auth composto por “matrícula + nome” deve trafegar explicitamente sobre a rede, sendo rastreável através de inspeção profunda de pacotes. Abaixo, reproduz-se o *hexdump* de uma captura real (`tcpdump -X`), demonstrando a presença legível da *magic string* RDFT e as credenciais, garantindo a conformidade total com o requisito de autenticação de carga útil.

```
0x0030:  2977 f920 5244 4654 0000 0000 0010 0000  )w..RDFT.....
0x0050:  0000 3230 3236 3130 3131 3431 3020 414e  ..20261011410.AN
0x0060:  5448 4f4e 5920 4952 4c41 4e20 4d41 5251  THONY.IRLAN.MARQ
0x0070:  5545 5320 4c55 5a00 0000 0000 0000 0000  UES.LUZ.....
```

Evidências suplementares (telas do Wireshark via filtro `frame contains "ANTHONY"`) e os respectivos arquivos `.pcap` foram anexados ao repositório institucional do projeto em `results/evidence/`. Os *logs* brutos completos da bateria (60 capturas `.pcap`, registros JSONL da aplicação e CSVs derivados, ≈ 155 MB), por não serem versionados no repositório, encontram-se disponíveis na nuvem para análise posterior².

5. Síntese e Respostas aos Requisitos

O que a aplicação mediu condiz com o `tcpdump`? Plenamente. Através da técnica de integração de dados provamos que a cronometragem converge (Figura 8) e que as instâncias de retransmissão escalam em ambas as topologias conforme previsto matematicamente (Figura 7). O R-UDP, ao contrário do TCP, possui o mérito de expor essa carga analítica diretamente para a aplicação.

²Pasta `logs/` completa: https://drive.google.com/drive/folders/12sBrG_SSxHayG23rZKkMgdw7EalQI_ml?usp=sharing

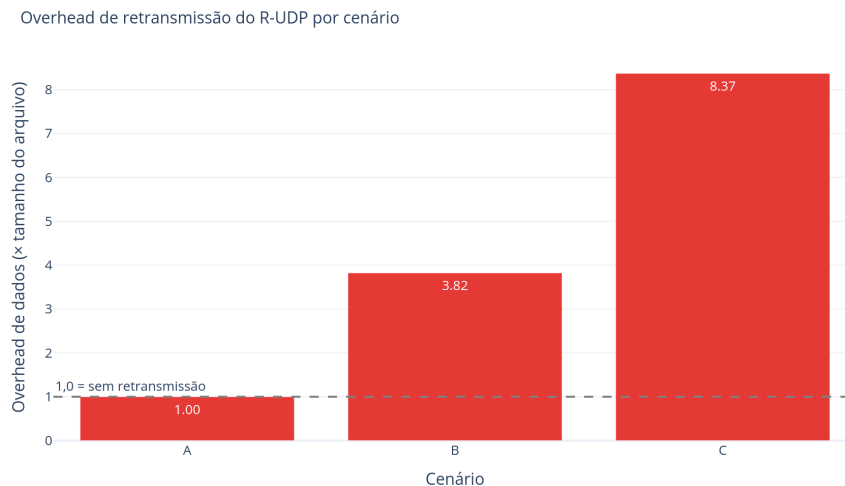


Figura 6. Overhead bruto de tráfego do R-UDP, evidenciando o custo da redundância inerente à estratégia Go-Back-N.

Qual o custo da confiabilidade em espaço de usuário? O custo mostrou-se condicionado à qualidade do canal. Em regime ótimo, a penalidade é negligenciável (fração de segundo). Em zonas de intenso descarte, o custo transpõe severamente em tempo (degradação de $1,9\times$ frente ao TCP no Cenário C) e em redundância de tráfego ($8,4\times$ mais envios). O TCP aparenta melhor resiliência em tempo pois converte a perda em estrangulamento direto de vazão, minimizando as rodadas cegas de tráfego injetado na rede.

O algoritmo Go-Back-N é adequado sob perda alta? A rigor, sua eficácia é questionável. Ele preserva o postulado da integridade (100% de sucesso demonstrado), mas aniquila o *goodput* da rede ao forçar a retransmissão bruta de segmentos validados. Conjugado à modelagem de amplificação do erro causada pela MTU, o Go-Back-N provou-se altamente inadequado para enlaces agressivos, abrindo precedentes inequívocos para a adoção mandatória de lógicas de *Selective Repeat*.

6. Conclusão e Trabalhos Futuros

Este trabalho materializou o dimensionamento empírico da confiabilidade de redes. Projetamos e instrumentamos arquiteturas de transporte em contêineres e validamos, com cruzamento de *logs* e topologias de interceptação (*tcpdump*), a exata assimetria comportamental entre o tratamento nativo e abstraído de erros (TCP) e a mecânica exposta em camada de aplicação (R-UDP / Go-Back-N). Isolamos e provamos matematicamente que o limite físico do enlace (MTU) atua como um severo catalisador de falhas para datagramas expandidos.

Para a próxima fase deste ciclo de pesquisa (Modelagem Estocástica), o arcabouço lógico do R-UDP aqui ratificado será traduzido para um simulador de eventos discretos fundamentado em *SimPy*. O modelo empírico sedimentado por este relatório proverá os parâmetros de base (*baselines*) que permitirão validar estatisticamente as projeções puramente teóricas simuladas em ambiente computacional no futuro.

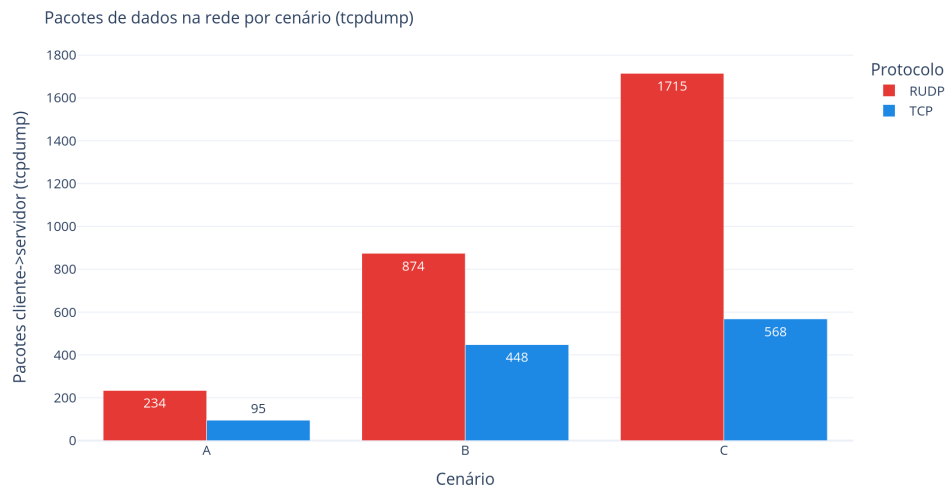


Figura 7. O comportamento oculto do TCP exposto: contagem bruta de tráfego cliente→servidor registrada pelo tcpdump.

Referências

- Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Hemminger, S. (2005). Network emulation with NetEm. In *Linux Conference Australia (LCA)*.
- Kurose, J. F. and Ross, K. W. (2017). *Computer Networking: A Top-Down Approach*. Pearson, 7th edition.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).
- Postel, J. (1980). User datagram protocol. Technical Report RFC 768, Internet Engineering Task Force (IETF).
- Postel, J. (1981). Transmission control protocol. Technical Report RFC 793, Internet Engineering Task Force (IETF).
- Tanenbaum, A. S. and Wetherall, D. J. (2011). *Computer Networks*. Pearson, 5th edition.

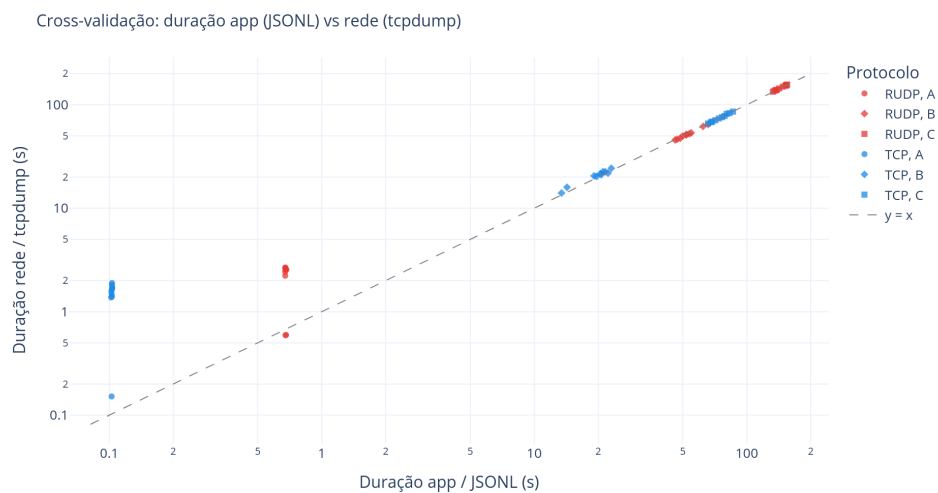


Figura 8. Dispersão para validação cruzada do tempo de transferência (Aplicação vs. `tcpdump`). A convergência para a bissetriz $y = x$ chancela a precisão dos registros internos.